STARSAI — Ingest + Preprocess Report

STARS AI
May 17, 2026

CONTENTS

STARSAI — Ingest + Preprocess Report

1. Outputs

1.1 Raw downloads — data/raw/

1.2 Bronze parquet — data/bronze/

2. Per-Dataset Preprocessing

2.1 TTC GTFS — stops.parquet, stop_times.parquet, trips.parquet, routes.parquet, calendar.parquet

2.2 MCI Crime — crime_night.parquet

2.3 311 Service Requests — service311.parquet

2.4 Lighting Poles — lighting_poles.parquet

2.5 OSM Ontario PBF — osm/toronto.osm.pbf

3. Blockers (autonomy report-only)

4. SDLC Alignment

5. Dependencies Installed

6. Files Created/Modified (uncommitted)

7. Next Steps (recommended order)

8. Reproducibility

STARSAI — Ingest + Preprocess Report

Run date: 2026-05-17 **Pipeline stages executed:** Ingest (raw download) → Preprocess (bronze parquet, factor-essential columns only) **Scope:** Datasets required by SDLC §5.1 factor catalogue (FR-001 through FR-017). Not yet executed: silver (CRS normalize, dedupe), gold (per-stop feature table), score (composite T-NTSI), pack (GeoJSON output).

---

1. Outputs

1.1 RAW DOWNLOADS — `DATA/RAW/`

Path	Bytes	Source
`gtfs/ttc/` (extracted)	34 MB zip	Toronto CKAN `7795b45e/cfb6b2b8`
`toronto/major-crime-indicators.csv`	126 MB	Toronto CKAN `6909ea60/8084057c`
`toronto/311-service-requests-2025.zip`	6.8 MB	Toronto CKAN `f3db05ab`
`toronto/311-service-requests-2024.zip`	5.8 MB	Toronto CKAN `f46b640d`
`toronto/topographic-poles.csv`	51 MB	Toronto CKAN `5ce5f150`
`osm/ontario-latest.osm.pbf`	894 MB	Geofabrik
`manifest.json`	—	URL + fetched_at + sha256 + bytes per file

1.2 BRONZE PARQUET — `DATA/BRONZE/`

File	Rows	Columns kept
`stops.parquet`	9,378	stop_id, stop_code, stop_name, stop_lat, stop_lon, location_type, parent_station, wheelchair_boarding, system, uid
`stop_times.parquet`	4,261,259	trip_id, arrival_time, departure_time, stop_id, stop_sequence, pickup_type, drop_off_type, system
`trips.parquet`	~210k (TTC)	route_id, service_id, trip_id, direction_id, shape_id, trip_headsign, system
`routes.parquet`	~200 (TTC)	route_id, route_short_name, route_long_name, route_type, agency_id, system
`calendar.parquet`	~50 (TTC)	service_id, monday…sunday, start_date, end_date, system
`crime_night.parquet`	37,367	event_id, occ_date, occ_hour, mci_category, offence, lat, lon
`service311.parquet`	32,123	created_date, request_type, lat, lon, ward, postal_fsa
`lighting_poles.parquet`	221,090	lat, lon, feature_type
`osm/toronto.osm.pbf`	—	326 MB clipped from 894 MB Ontario PBF
`manifest.json`	—	row counts + pull_date + generated_at

2. Per-Dataset Preprocessing

2.1 TTC GTFS — `STOPS.PARQUET`, `STOP_TIMES.PARQUET`, `TRIPS.PARQUET`, `ROUTES.PARQUET`, `CALENDAR.PARQUET`

SDLC mapping: FR-001 (TTC GTFS ingest), FR-012 (wait-exposure factor)

Columns kept (per file):

- `stops.txt` → stop_id, stop_code, stop_name, stop_lat, stop_lon, location_type, parent_station, wheelchair_boarding
- `stop_times.txt` → trip_id, arrival_time, departure_time, stop_id, stop_sequence, pickup_type, drop_off_type
- `trips.txt` → route_id, service_id, trip_id, direction_id, shape_id, trip_headsign
- `routes.txt` → route_id, route_short_name, route_long_name, route_type, agency_id
- `calendar.txt` → service_id, monday–sunday flags, start_date, end_date

Columns dropped: stop_desc, zone_id, stop_url, stop_timezone (stops); stop_headsign, shape_dist_traveled, timepoint (stop_times); block_id, wheelchair_accessible, bikes_allowed, trip_short_name (trips); route_desc, route_url, route_color, route_text_color, route_sort_order (routes).

Row filters: Drop rows where `stop_lat` or `stop_lon` is null/non-numeric.

Augmented: Added `system = "ttc"` column on every table. Added `uid = "ttc:" + stop_id` to `stops.parquet` for cross-system joins (currently TTC-only but future-proof).

2.2 MCI CRIME — `CRIME_NIGHT.PARQUET`

SDLC mapping: FR-002, FR-009 (crime-history factor — decay-weighted incidents within 250m, last 36 months, 9pm–4am only)

Columns kept (7): event_id, occ_date, occ_hour, mci_category, offence, lat, lon

Columns dropped (~28): REPORT_DATE, OCC_YEAR, OCC_MONTH, OCC_DOW, OCC_DOY, OCC_DAY, REPORT_YEAR, REPORT_MONTH, REPORT_DOW, REPORT_DOY, REPORT_DAY, REPORT_HOUR, LOCATION_TYPE, PREMISES_TYPE, UCR_CODE, UCR_EXT, DIVISION, HOOD_158, NEIGHBOURHOOD_158, HOOD_140, NEIGHBOURHOOD_140, X, Y (projected coords — kept WGS84 only), OBJECTID.

Row filters:

- `occ_hour ∈ {21, 22, 23, 0, 1, 2, 3}` (night window per FR-002)
- `occ_date >= 2023-05-17` (last 36 months from pull_date)
- `lat ∈ [43.55, 43.90]`, `lon ∈ [-79.70, -79.10]` (Toronto bounding box sanity)
- `lat`, `lon` not null

Rationale: Daytime incidents and rows outside the Toronto bbox carry no signal for the nighttime safety index and inflate compute cost on downstream H3 binning + buffer joins. The MCI dataset is known to have a privacy-offset of ~100m on lat/lon — this is accepted and documented as risk R-01 in the SDLC; the 250m buffer in FR-009 absorbs the offset.

2.3 311 SERVICE REQUESTS — `SERVICE311.PARQUET`

SDLC mapping: FR-005, FR-016 (311 disorder factor — counts of disorder requests in last 90 days within 200m, but ingest pulls 12mo to support seasonality analysis later)

Source: 2024 + 2025 ZIP archives, each containing yearly CSV. Concatenated.

Columns kept (6): created_date, request_type, lat, lon, ward, postal_fsa

Columns dropped: Division, Section, Status, Intersection Street 1, Intersection Street 2, plus year-specific admin fields that vary across CSV vintages.

Row filters:

- `created_date >= 2025-05-17` (last 12 months)
- `request_type` (case-insensitive) contains any of: `graffiti`, `noise`, `drug`, `needle`, `syringe`, `dumping`, `encampment`, `vandalism`, `litter`, `human waste` — matches FR-016 disorder typology

Parser hardening: Yearly CSV column counts vary; used `engine="python"`, `on_bad_lines="skip"`, multi-encoding fallback (`utf-8` → `utf-8-sig` → `latin-1`) to absorb upstream irregularities without crashing.

2.4 LIGHTING POLES — `LIGHTING_POLES.PARQUET`

SDLC mapping: FR-003, FR-008 (lighting-density factor — log-scaled count within 75m buffer)

Source schema: `_id, SUBTYPE_CODE, SUBTYPE_DESC, ELEVATION, DERIVED_HEIGHT, LAST_GEOMETRY_MAINT, LAST_ATTRIBUTE_MAINT, OBJECTID, geometry` — geometry is GeoJSON `MultiPoint` (not Point).

Columns kept (3): lat, lon, feature_type (= SUBTYPE_DESC)

Columns dropped: _id, SUBTYPE_CODE, ELEVATION, DERIVED_HEIGHT, LAST_GEOMETRY_MAINT, LAST_ATTRIBUTE_MAINT, OBJECTID, raw geometry string.

Row filters:

- Toronto bbox sanity (`lat ∈ [43.55, 43.90]`, `lon ∈ [-79.70, -79.10]`)
- `SUBTYPE_DESC` regex `LIGHT|LAMP|STREET|UTIL|HYDRO` — keeps “Street Light Pole”, “Utility Pole”, drops decorative or signage poles. **221,090 rows retained.**

Geometry parser: Reads first coordinate of `MultiPoint` (always a single point in this dataset despite the type label). Falls back to `Point` if encountered.

2.5 OSM ONTARIO PBF — `OSM/TORONTO.OSM.PBF`

SDLC mapping: FR-004 (OSM POI ingest), feeds FR-010 eyes-on-street, FR-011 isolation, FR-013 sightline, FR-015 crossing-danger.

Action: Bounding-box clip via `pyosmium` (Python binding). Toronto bbox `(-79.6394, 43.5781, -79.1163, 43.8554)` applied to nodes. Ways and relations passed through (downstream filtering by node membership is cheaper than re-deriving them here).

Size reduction: 894 MB → 326 MB (~64% reduction). Tag set unchanged — every OSM tag is preserved in the clipped PBF so silver/gold stages can filter by `amenity`, `opening_hours`, `building`, `highway`, `lit`, etc. without re-downloading.

Why not parquet at this stage: PBF is the native, indexable format for spatial filtering. Converting to parquet too early loses the `osmium` query primitives. Tag extraction to parquet happens in the silver stage when factor needs are concrete.

---

3. Blockers (autonomy report-only)

Source	Failure	Status
MiWay GTFS	`https://www.miway.ca/gtfs/google_transit.zip` returns 0-byte response (hotlink-blocked)	Disabled in `GTFS_URLS_DISABLED`. Future: scrape from Mississauga Open Data portal.
Brampton GTFS	`https://www.brampton.ca/.../Google_Transit.zip` → 404	Disabled. Future: find new resource ID.
YRT GTFS	`https://www.yrt.ca/.../google_transit.zip` → 502 (upstream down)	Disabled. Future: retry or use transitfeeds aggregator.
Halton GTFS	`https://www.halton.ca/Repository/Google_Transit.zip` → 404	Disabled.
StatCan 2021 Census income (DS-09)	Not in `config.py`. SDLC FR-006 requires it for equity layer.	Defer to next sprint — needs StatCan Web Data Service integration (separate API).
Mapillary (DS-12), AGCO (DS-11), CCTV (DS-05), KSI (DS-07), Neighbourhoods (DS-08)	Not yet sourced	All P1/P2 per SDLC §2.1. Schedule in sprint 2.

Impact: SDLC success metric “≥12,000 stops scored” depends on regional feeds. TTC alone yields 9,378 stops. Either re-source MiWay/Brampton/YRT/Halton or rescope to “TTC-only nighttime safety index” for v1.0.

---

4. SDLC Alignment

Concern	SDLC ref	Status
Folder layout `data/{raw,bronze,silver,gold,dist}`	Appendix A	✅ raw + bronze populated; silver/gold/dist pending
Parquet storage for tabular	§3.3, §4.2	✅ all tabular bronze outputs are parquet (Snappy)
H3 r10 spatial index	§4.3	⏳ deferred to transform/gold stage
Manifest with URL + fetched_at + sha256	§4.2 ingest stage	✅ `data/raw/manifest.json`
Pandera schema validation	§8.3	⏳ schema not yet defined; bronze is intentionally schema-tolerant
`make all` reproducibility	FR-034, §7.3	⏳ Makefile not yet created
Python 3.12 + uv	§3.3, NFR-017	⚠️ used Python 3.11.9 (pre-installed); venv via `py -m venv`. Documented divergence.
Pinned deps	NFR-017	⚠️ deps installed via `pip install` not `uv sync`. `requirements.txt` not regenerated.

5. Dependencies Installed

Pipeline venv: `pipeline/.venv/` (Python 3.11.9). Installed (vs. `requirements.txt` request):

Package	Version	Why
requests	2.x	HTTP download
pandas	2.x	CSV parsing, parquet write
pyarrow	implicit via pandas	Parquet engine
shapely	2.x	(declared, unused in this stage; reserved for silver)
geopandas	1.1.3	(declared, unused; reserved for silver)
osmium	4.3.1	OSM PBF bbox clip — pyosmium binding (replaces `pyrosm` which fails to build on Windows Python 3.11)
tqdm, urllib3, certifi, etc.	—	Transitive

Skipped (not needed for ingest/preprocess): `duckdb`, `h3`, `xgboost`, `scikit-learn`, `librosa`, `scipy`, `matplotlib`, `seaborn`, `jupyter`, `ultralytics`, `mediapipe`, `opencv-python`. Add when factor compute / score / model stages begin.

Substitution: `pyrosm` (declared in `requirements.txt`) → `osmium`. Reason: pyrosm wheel build failure on Windows + Python 3.11.9. `osmium` provides equivalent PBF reading + a cleaner bbox-filter handler API. **Action required:** update `requirements.txt` to pin `osmium` instead of `pyrosm`.

---

6. Files Created/Modified (uncommitted)

File	Action	Purpose
`pipeline/ingest.py`	rewritten	Proper retries, multi-source download, manifest emission
`pipeline/preprocess.py`	new	Bronze stage — strip cols, filter rows, write parquet
`pipeline/config.py`	edited	Fix TTC GTFS URL; switch MCI to `ckan_file`; move broken regional feeds to `GTFS_URLS_DISABLED`
`pipeline/.venv/`	new	Local venv (gitignored)
`data/raw/**`	new	Raw downloads (gitignored)
`data/bronze/**`	new	Bronze parquet outputs (gitignored)

Not committed per user directive.

---

7. Next Steps (recommended order)

1. **Re-source regional GTFS** or rescope index to TTC-only.
2. **StatCan 2021 income API** integration (FR-006 equity layer).
3. **Silver stage** — CRS normalize to EPSG:4326 (already mostly compliant), dedupe MCI by `event_id`, time normalize to UTC + derive `local_hour_toronto`. Best fit for parallel agents (one per dataset).
4. **OSM tag extraction** — silver-stage: amenity + opening_hours → `osm_pois.parquet`; building footprints → `osm_buildings.parquet`; highway with `lit=*` → `osm_lit_ways.parquet`.
5. **H3 r10 grid generation** (transform stage) over Toronto bbox.
6. **Pandera schemas** for each bronze + silver file; wire into `make validate`.
7. **`Makefile` + `make all`** target — FR-034 reproducibility gate.

---

8. Reproducibility

```
# From repo root
cd pipeline
py -m venv .venv
.venv\Scripts\python.exe -m pip install requests pandas pyarrow shapely geopandas osmium tqdm
.venv\Scripts\python.exe ingest.py
.venv\Scripts\python.exe preprocess.py
```

Outputs land in `data/raw/` and `data/bronze/`. Both stages are resumable — re-running skips already-downloaded files.